CO1 – ASSESSMENT TOOL 1

ANALYTICAL PROBLEM SOLVING

Name	Rishi Varthini S
Registration Number	192421304
Course Code	CSA1735
Course Name	Artificial Intelligence
Faculty Name	Dr. R. Thalapathi Rajasekaran

INTRODUCTION

Artificial Intelligence (AI) enables computer systems to solve problems by representing states, selecting actions, searching alternatives, and evaluating possible solutions. Search algorithms are used in puzzles, route planning, logistics, and automated booking systems.

This assignment explains five AI problem-solving scenarios: the 8-puzzle using heuristic search, food-delivery route optimisation using local search, comparison of BFS, DFS, and IDS, an OLA-style cab-booking agent with pseudocode, and logistics route planning using Uniform Cost Search.

OBJECTIVES

- Represent real-world tasks using states, actions, goals, and costs.
- Understand how heuristics guide informed search.
- Apply local search to improve a delivery route.
- Compare uninformed search methods by completeness, optimality, time, and space.
- Design a problem-solving agent and express its behaviour in pseudocode.
- Find a least-cost route in a weighted network.

1. 8-PUZZLE WITH HEURISTIC SEARCH

1.1 Problem Statement

The 8-puzzle is a 3 × 3 board containing eight numbered tiles and one blank space. A tile next to the blank may slide into it. The goal is to reach a target arrangement using the fewest moves possible.

1.2 Problem Formulation

- State: current arrangement of tiles and blank.
- Initial state: starting board configuration.
- Actions: move the blank up, down, left, or right when legal.
- Transition model: swap the blank with an adjacent tile.
- Goal test: current board matches the target arrangement.
- Path cost: each move costs 1; total cost equals the number of moves.

1.3 Example

Initial state:

```
1 2 3
4   6
7 5 8
```

Goal state:

```
1 2 3
4 5 6
7 8 _
```

The symbol "_" represents the blank. Move the blank down to swap with 5, then right to swap with 8. The goal is reached in two moves.

1.4 Heuristic Functions and A*

A heuristic h(n) estimates the remaining cost from state n to the goal. A* selects states using f(n) = g(n) + h(n), where g(n) is the cost already incurred.

Heuristic	Definition	Property
Misplaced tiles	Counts numbered tiles not in their goal positions; ignores blank.	Simple and admissible.
Manhattan distance	For each numbered tile, adds |current row − goal row| + |current column − goal column|.	Usually more informative and admissible.

An admissible heuristic never overestimates the true minimum remaining cost. Both heuristics are admissible for the standard 8-puzzle when every move costs 1. Manhattan distance usually guides the search better because it measures how far each tile is from its goal.

1.5 A* Pseudocode

```
A  STAR(start, goal):
   open_set = priority queue ordered by f(n) = g(n) + h(n)
   g_score[start] = 0
   insert start with priority h(start)
   parent[start] = NONE

   while open_set is not empty:
      current = remove state with lowest priority
      if current == goal:
         return reconstruct_path(parent, current)

      for each legal neighbour of current:
         tentative  g = g  score[current] + 1
         if neighbour is new OR tentative_g < g_score[neighbour]:
            parent[neighbour] = current
            g  score[neighbour] = tentative  g
            priority = tentative_g + h(neighbour)
            insert/update neighbour in open  set
   return FAILURE
```

1.6 Advantages and Limitations

- A* can find an optimal solution with an admissible heuristic and a correct implementation.
- A useful heuristic can reduce the number of explored states.
- A* may consume considerable memory to store generated states and costs.

- A weak heuristic provides little guidance and may explore many states.

1.7 Conclusion

The 8-puzzle demonstrates informed search. A* combines the cost already incurred with an estimate of the remaining cost to prioritise promising states and find a shortest solution under suitable conditions.

2. FOOD DELIVERY ROUTE OPTIMISATION (LOCAL SEARCH)

2.1 Problem Statement

A food-delivery service must deliver orders from a restaurant to multiple customers. The objective is to arrange the stops to reduce total distance or travel time while satisfying constraints such as delivery deadlines and vehicle capacity.

2.2 Representation

- Candidate solution: an ordered route such as Restaurant → A → B → C → Restaurant.
- Objective: minimise the sum of travel distances or estimated travel times between consecutive stops.
- Constraints: visit every assigned customer and respect capacity and delivery windows.
- Neighbourhood: swap two stops, reverse a route segment, or relocate one stop.

2.3 Local Search Method

Local search begins with a valid route and repeatedly examines nearby routes. If a neighbour has a lower objective value, it replaces the current route. The 2-opt method reverses a segment of the route and can remove inefficient crossings. Search stops when no tested neighbour improves the solution or a preset limit is reached.

2.4 Illustrative Example

Suppose a restaurant R must deliver to A, B, C, and D. The initial route is R → A → B → C → D → R. Assume the estimated distances are:

Route segment	Distance (km)
R → A	4
A → B	8
B → C	3
C → D	7
D → R	6
Total	28

A neighbour such as R → A → C → B → D → R can be evaluated using the road-distance matrix. If its total distance is smaller and it satisfies the constraints, the route is accepted. The values above define only the initial route; the alternative must be calculated from actual route distances.

2.5 Hill-Climbing Pseudocode

```
LOCAL_SEARCH(initial_route):
    current = initial_route
    current_cost = route_cost(current)

    repeat:
        best_neighbour = current
        best  cost = current  cost
        for each neighbour in generate_neighbours(current):
            if neighbour satisfies all constraints:
                cost = route_cost(neighbour)
                if cost < best_cost:
                    best_neighbour = neighbour
                    best_cost = cost
        if best_cost >= current_cost:
            return current
        current = best  neighbour
        current_cost = best_cost
```

2.6 Evaluation, Advantages and Limitations

- Total distance: sum of distances between consecutive stops.
- Delivery time: driving, service, and waiting time.
- On-time delivery rate: percentage of orders delivered within the promised time.
- Local search improves routes without examining every possible stop order.
- Basic hill climbing can stop at a local optimum instead of the global optimum.
- Traffic changes, new orders, and delivery time windows require updated costs and constraint checks.

2.7 Conclusion

Local search improves a delivery route by comparing nearby alternatives. It is practical for large route spaces, although simulated annealing, tabu search, or genetic algorithms may help escape poor local optima.

3. UNINFORMED SEARCH COMPARISON (BFS, DFS, IDS)

3.1 Introduction

Uninformed search explores a state space without a heuristic estimate of distance to the goal. BFS, DFS, and IDS differ in the order in which they explore nodes and in their memory requirements.

3.2 Breadth-First Search (BFS)

BFS explores nodes level by level and uses a FIFO queue.

```
BFS(start, goal):
    queue = [start]
```

```
    visited = {start}
    while queue is not empty:
        node = remove_front(queue)
        if node == goal:
            return path_to(node)
        for each child of node:
            if child not in visited:
                visited.add(child)
                parent[child] = node
                add child to back of queue
    return FAILURE
```

3.3 Depth-First Search (DFS)

DFS follows one branch as deeply as possible before backtracking. It uses a stack or recursion. Cycle checking is important for graph searches.

```
DFS(start, goal):
    stack = [start]
    visited = empty set
    while stack is not empty:
        node = remove  top(stack)
        if node == goal:
            return path  to(node)
        if node not in visited:
            visited.add(node)
            for each child of node:
                if child not in visited:
                    parent[child] = node
                    push child onto stack
    return FAILURE
```

3.4 Iterative Deepening Search (IDS)

IDS runs depth-limited DFS repeatedly, starting with limit 0 and increasing the limit by 1 each time. It combines DFS's low memory use with BFS's shallow-solution guarantee when the branching factor is finite and step costs are equal.

```
IDS(start, goal):
    for limit = 0, 1, 2, ...:
        result = DEPTH  LIMITED  DFS(start, goal, limit)
        if result is a solution:
            return result

DEPTH_LIMITED_DFS(node, goal, limit):
    if node == goal: return solution
    if limit == 0: return CUTOFF
    for each child of node:
        result = DEPTH_LIMITED_DFS(child, goal, limit - 1)
        if result is a solution: return result
    return FAILURE
```

3.5 Comparison

Criterion	BFS	DFS	IDS
Node selection	Shallowest first	Deepest current branch	Depth limit increases
Data structure	FIFO queue	Stack / recursion	Repeated depth-limited

			DFS
Complete?	Yes, finite branching	Not in infinite-depth spaces; yes for finite graphs with cycle handling	Yes, finite branching
Optimal?	Yes, equal step costs	No, in general	Yes, equal step costs
Time	O(b^(d+1))	O(b^m)	O(b^d)
Space	O(b^(d+1))	O(bm)	O(bd)
Best use	Shallowest solution needed	Memory limited; depth manageable	Depth unknown; memory limited

Notation: b is the branching factor, d is the depth of the shallowest goal, and m is the maximum search depth. Complexity depends on the graph representation and implementation.

3.6 Conclusion

BFS is suitable when a shallowest solution is required and memory is available. DFS uses less memory but may follow a long branch. IDS is a useful compromise when solution depth is unknown and every action has equal cost.

4. OLA CAB BOOKING (PROBLEM-SOLVING AGENT AND PSEUDOCODE)

4.1 Problem Statement

An OLA-style cab-booking system receives a ride request and selects a suitable driver. It considers pickup distance, estimated arrival time, driver availability, vehicle type, and estimated fare. The goal is to assign a feasible driver and confirm the booking.

4.2 Problem Formulation

Element	Description
Initial state	Passenger submits pickup, destination, and preferences.
State space	Available drivers, locations, vehicle types, assignment status.
Actions	Validate request, filter drivers, estimate pickup times, rank candidates, send offers.
Transition model	Driver accepts or rejects; availability can change.
Goal test	Suitable driver accepts and booking is confirmed.
Action cost	Weighted score based on pickup time, distance, fare, and preference match.

4.3 Agent Design

This can be modelled as a goal-based problem-solving agent. It observes the request and driver data, formulates the goal, finds eligible drivers, compares candidates, and attempts the best feasible assignment. A real system must also handle traffic, cancellations, driver consent, and privacy.

4.4 Driver Selection Score

```
Score(driver) = w1 × estimated_pickup_time
              + w2 × pickup_distance
              + w3 × estimated_fare
              + w4 × preference_penalty
```

The weights are non-negative values chosen by the service. A lower score is preferred. Unavailable drivers and drivers who fail mandatory requirements are excluded before ranking.

4.5 Pseudocode

```
OLA_BOOKING_AGENT(request, drivers):
    if pickup or destination is missing:
        return "Ask passenger to complete the request"

    eligible = empty list
    for each driver in drivers:
        if driver is available AND vehicle matches request:
            eta = estimate_pickup_time(driver.location, request.pickup)
            score = calculate_score(eta,
                    distance(driver.location, request.pickup),
                    estimate_fare(request),
                    preference_penalty(driver, request))
            add (driver, score) to eligible

    if eligible is empty:
        return "No suitable driver available"

    sort eligible by score ascending
    for each (driver, score) in eligible:
        send booking offer to driver
        if driver accepts:
            mark driver as assigned
            create booking ID
            notify passenger with driver and ETA
            return "Booking confirmed"

    return "No driver accepted; offer retry or cancellation
```

4.6 Example, Advantages and Limitations

Suppose three eligible drivers have estimated pickup times of 4, 7, and 5 minutes. If other factors are equal, the first driver is ranked highest. If that driver rejects the offer, the system tries the next candidate.

- Automates driver selection and reduces manual work.
- Multiple factors can be considered instead of distance alone.
- Availability and traffic data can become outdated and should be refreshed.

- The lowest-score driver may reject; the agent must handle retries safely.

4.7 Conclusion

The cab-booking agent demonstrates goal-directed problem solving. Filtering eligible drivers, ranking candidates, and handling acceptance or rejection helps make booking decisions systematically.

5. LOGISTICS DELIVERY NETWORK (UNIFORM COST SEARCH)

5.1 Problem Statement

A logistics company must transport a package from a source warehouse to a destination through warehouses and distribution centres. Each connection has a cost, such as distance, time, or expense. Uniform Cost Search (UCS) finds a least-cost route by expanding the path with the lowest cumulative cost.

5.2 Graph Representation

Represent the network as a weighted graph G = (V, E). Vertices represent warehouses or distribution centres, and edges represent routes with non-negative costs. The path cost g(n) is the sum of edge costs from the source to node n.

5.3 Example Network

Connection	Cost
A → B	2
A → C	5
B → C	1
B → D	4
C → D	1
C → E	5
D → E	2

A is the source and E is the destination. Route A → B → C → D → E costs 2 + 1 + 1 + 2 = 6. Route A → C → E costs 5 + 5 = 10. Therefore, the first route is cheaper among these alternatives.

5.4 Uniform Cost Search Pseudocode

```
UNIFORM_COST_SEARCH(start, goal):
    frontier = priority queue
    insert (cost = 0, start) into frontier
    best  cost[start] = 0
    parent[start] = NONE

    while frontier is not empty:
        (cost, node) = remove entry with lowest cost
        if cost > best_cost[node]:
```

```
        continue
    if node == goal:
        return reconstruct_path(parent, goal), cost

    for each (neighbour, edge  cost) from node:
        new  cost = cost + edge  cost
        if neighbour is new OR new_cost < best_cost[neighbour]:
            best  cost[neighbour] = new  cost
            parent[neighbour] = node
            insert (new  cost, neighbour) into frontier
return FAILURE
```

5.5 Step-by-Step Execution

1. Start at A with cumulative cost 0.
2. Expand A: add B with cost 2 and C with cost 5. The queue selects B next.
3. Expand B at cost 2: update C to cost 3 through B and add D with cost 6.
4. Expand C at cost 3: update D to cost 4 through C and add E with cost 8.
5. Expand D at cost 4: update E to cost 6 through D.
6. Remove E from the priority queue at cost 6. The least-cost route is A → B → C → D → E.

5.6 Advantages and Limitations

- UCS finds an optimal least-cost route when edge costs are non-negative and the algorithm is implemented correctly.
- It supports different costs for different roads, unlike BFS which treats each step equally.
- It may require considerable time and memory on large networks.
- Negative edge costs are unsuitable for standard UCS; changing traffic may require rerunning the search.

5.7 Conclusion

UCS is suitable for logistics networks where connections have different costs. By expanding the least cumulative-cost path first, it finds an optimal route when edge costs are non-negative.